Detailed Planning Memo

Async Context Communication, ChannelGraph Recovery, and Sink→Source Tunnel Detection

Expanded from the earlier sections 4–9 analysis, aligned with update_0305, the unstripped ELF evaluation report, and RQ1 proposal v1.3

Executive summary

This memo expands the next-step design after source and sink labeling. The current SourceAgent implementation already provides a strong evidence-grounded RQ1 layer: it can label hardware-dependent sources and dangerous sinks, and the current unstripped ELF suite reports 48 samples, 3588 total detections, and strict sink GT results of TP=17, FP=0, FN=0. The missing capability is not "finding more isolated labels"; it is connecting them across asynchronous context boundaries to produce actionable vulnerability chains.

What is already strong	**Context-aware source labeling (MMIO, ISR, DMA), staged sink labeling (copy, memset, store, loop, func-ptr, format), evidence packs, fail-closed verifier, and facts export for BinAgent.**
What is still missing	Async communication recovery (ChannelGraph), parse/derive facts, check summarization, and a tunnel-aware source→sink linker.
Central design decision	Do not enumerate all source×sink pairs. Use a Mango-style sink-first heuristic linker, and only jump across a small number of recovered async-channel edges.
Main recommendation	Implement a ChannelGraph MVP first (shared SRAM objects + DMA), then add sink-first tunnel detection for len/index/format roots, and use LLM only for suspicious semantic cases such as parse logic and check adequacy.

Current capability	Status	Why it matters now
Source miners (MMIO/ISR/DMA)	Implemented and already useful	These are the strongest firmware-specific facts and are the main reason the project is not just another Type-I taint analyzer.
Sink miners (copy/memset/store/loop/func-ptr/format)	Implemented, but chain-unaware	They identify where dangerous effects may happen, but do not yet prove that attacker-controlled input reaches them.
Verifier obligations	Implemented	This is the fail-closed trust anchor. It prevents overclaiming and should remain the final

		arbiter.
ChannelGraph	Not yet implemented	This is the Type II/III equivalent of IPC modeling. Without it, cross-context chains remain disconnected.
Parse/derive and check summarization	Not yet implemented as first-class facts	These are the semantic bridges from raw bytes to dangerous arguments like len/index/format.
Sink-first tunnel linker	Not yet implemented	This is the mechanism that turns labels into vulnerability findings.

4. Async context communication and ChannelGraph recovery (expanded)

The next major module is not another miner; it is an explicit model of how data crosses execution contexts in Type II/III firmware. In Linux-like Type-I systems, this role is often played by explicit IPC objects (socket, pipe, message queue, RPC channel). In Type II/III firmware, there is usually a single address space, so the same role is played by shared objects and event mechanisms: buffers, head/tail indices, state flags, semaphores, queues, and DMA destination buffers.

Why this module is necessary:

- Without a communication graph, source labels and sink labels remain isolated facts. This is enough for semantic recovery, but not enough for end-to-end vulnerability claims.
- Cross-context flows often do not follow an ordinary call chain. The producer may be an ISR or DMA engine, while the consumer may be a task or the main loop; a classic inter-procedural slice cannot cross this boundary by itself.
- A graph representation lets the linker “tunnel” across contexts in a disciplined way instead of trying to model full concurrency or scheduling.

The best way to think about ChannelGraph is: it is the Type II/III equivalent of the IPC graph that tools like KARONTE build for Type-I multi-binary firmware. KARONTE models taint propagation across binaries via explicit communication points; our ChannelGraph models taint propagation across ISR/task/main/DMA contexts via shared objects and synchronization artifacts.

4.1 What should be a node or edge in ChannelGraph?

Element	Examples	Why it exists	Benefit	Main limitation
Context node	ISR, task, main loop, workqueue, DMA-producer	Represents “who executes” and where an object is read or written.	Lets the linker reason about cross-context hops instead of ordinary call edges.	Task classification is hard in stripped binaries; ISR tagging is much easier than task tagging.
Object node	rx_buf, hdr, payload, ring buffer, rx_ready flag, rx_len, queue handle	Represents “what carries data” across contexts.	Natural place to attach evidence: address range, readers, writers, size, cluster.	Different objects may alias to the same RAM region; clustering can over-merge.

Producer edge	ISR writes hdr, DMA writes dma_buf, xQueueSendFromISR(q)	Shows where tainted data enters a channel object.	Allows source-side provenance to be preserved.	In some designs the producer only raises an event; the payload lives elsewhere.
Consumer edge	task reads rx_buf, main loop checks flag, queue receive in worker	Shows where data or events leave the channel object.	Creates the landing point for sink-first tunnel jumps.	Consumer may be a parser far from the actual receive site; ranking is needed.
Control/sync edge	semaphore give/take, flag set/clear, callback registration	Captures wake-up ordering when data and control are split.	Improves confidence that the producer→consumer relation is real.	Hard to recover generically in stripped RTOS code.

4.2 Which channels should be implemented first?

The ChannelGraph should be built in layers. The goal is not to model every RTOS primitive from day one. The goal is to cover the communication forms that are both common and statically recoverable.

4.2.1 Layer A: shared SRAM objects (recommended MVP)

This should be the first implementation target. It covers the most common and easiest-to-recover Type II/III channels: ISR-written global buffers, head/tail ring buffers, flags, length fields, and simple shared structs.

Representative example (UART ring buffer):

```
volatile uint8_t rx_buf[256];
volatile uint16_t head, tail;

void USART_IRQHandler(void) {
  rx_buf[head] = UART_DR;     // producer write
  head = (head + 1) & 0xff;   // ring update
}

void parser_task(void) {
  while (tail != head) {
    uint8_t b = rx_buf[tail];
    tail = (tail + 1) & 0xff; // consumer read
    parse_byte(b);
  }
}
```

- Reason to start here: MAI already indexes loads/stores, ISR-vs-non-ISR is already partially available, and SRAM objects are visible without requiring strong type recovery.
- Benefit: high coverage for real embedded firmware. Many UART/SPI/USB/ETH receive paths boil down to “shared RAM object + event flag”.
- Main defect: clustering SRAM regions can merge unrelated objects, especially in stripped binaries or structure-heavy code. This must be handled with conservative confidence and top-K ranking.

4.2.2 Layer B: DMA-backed objects

DMA is unusual because the producer is hardware: the firmware may never execute a CPU store into the destination buffer. The object must be recovered from DMA configuration writes rather than from ordinary RAM writes.

Representative example (DMA RX buffer):

```
void dma_init(void) {
    DMA_CPAR = UART_DR_ADDR;
    DMA_CMAR = &dma_rx_buf[0];   // producer object identified here
    DMA_CNDTR = 64;
}

void DMA_IRQHandler(void) {
    dma_done = 1;
}

void net_task(void) {
    if (dma_done) {
        process_packet(dma_rx_buf, 64);  // consumer side
    }
}
```

- Reason to implement next: the project already has DMA_BACKED_BUFFER evidence. Turning that evidence into graph edges is a natural extension instead of a new research direction.
- Benefit: this covers an important class of firmware that static source-only MMIO modeling misses, because the payload enters memory without a visible load instruction.
- Main defect: DMA controller variants and descriptor formats differ significantly. The first version should focus on the “simple config sequence” cases already covered by current DMA miners.

4.2.3 Layer C: RTOS queues, mailboxes, and workqueues

These are semantically very attractive because they are explicit producer–consumer channels, but they should come after shared SRAM and DMA. In stripped binaries they are harder to recognize than they first appear.

Representative example (FreeRTOS queue):

```
void SPI_IRQHandler(void) {
    msg.len = SPI_DR;
    xQueueSendFromISR(rx_q, &msg, &woken);
}

void parser_task(void) {
    xQueueReceive(rx_q, &msg, portMAX_DELAY);
    memcpy(dst, msg.payload, msg.len);
}
```

- Reason not to start here: symbol-less RTOS internals are much harder than user-space libc calls. A naive queue recognizer will quickly become brittle and architecture/RTOS specific.
- Benefit once available: explicit queue handles create very clean object nodes and often reduce false positives compared with broad SRAM clustering.

- Main defect: without symbols, queue APIs look like ordinary function calls and the payload layout may be hidden inside copied structs. This is a good target for later heuristics or LLM-assisted semantic classification, not for the MVP.

4.3 Recommended MVP detection rules

The MVP should deliberately bias toward precision and reuse existing evidence. A practical first version can follow the rules below.

1. MVP Rule 1 — Shared-object channel: create an object node for an SRAM cluster if it has at least one ISR writer and at least one non-ISR reader, and if the consumer side appears near parsing or sink activity.
2. MVP Rule 2 — Flag channel: treat a small scalar object as a control channel if one side writes constants like 0/1 and the other side uses it in if/while conditions.
3. MVP Rule 3 — DMA channel: whenever DMA_BACKED_BUFFER is verified, create a DMA→object edge immediately, and look for non-DMA consumers later.
4. MVP Rule 4 — Queue channel (symbolic only): only add queue/mailbox/workqueue edges when strong symbol evidence exists. Do not guess them from arbitrary list operations in the first implementation.

Why this ordering is good:

- It reuses what the current pipeline already knows: SRAM ranges, DMA candidates, ISR tagging, and decompilation artifacts.
- It keeps the graph sparse enough that sink-first linking remains tractable.
- It avoids the biggest risk of RQ2: overbuilding a generic RTOS recognizer before the basic cross-context graph has demonstrated value on real examples.

5. How to construct ChannelGraph in practice (detailed implementation plan)

This section turns the high-level idea into an implementable algorithm. The purpose is to make the next engineering step concrete enough that it can be assigned as work, unit-tested, and evaluated.

5.1 Inputs that should feed ChannelGraph

Input	What it already gives us	Why it is enough for the MVP	What it does not yet solve
MemoryAccessIndex	Function, context (ISR/non-ISR), load/store kind, target address, provenance, cluster	This is enough to spot shared SRAM objects and basic producer/consumer evidence.	It does not by itself tell us that a variable is semantically “len” or “rx_ready”.
Verified source labels	MMIO_READ, ISR_MMIO_READ, ISR_FILLED_BUFFER, DMA_BACKED_BUFFER	These supply trusted source-side anchors and reduce the need to rediscover hardware input from scratch.	They do not tell us how far the source flows after entering RAM.

Verified sink labels	COPY/MEMSET/STORE/LOOP/FUNC_PTR/FORMAT callsites	These provide the endpoints that will seed the linker.	They do not establish whether a dangerous argument is input-derived.
Decompile cache / CFG facts	Readable pseudocode, loop structure, xrefs, basic block relations	Useful for ranking candidate consumer functions and recognizing flag/queue patterns.	Decompiler text is noisy; it should stay secondary to IR-based facts.

5.2 Proposed object schema

```
ObjectNode {
  object_id,
  region_kind: SRAM_CLUSTER | DMA_BUFFER | QUEUE_HANDLE | FLAG | UNKNOWN,
  addr_range,
  evidence_refs,
  writers: [func_id],
  readers: [func_id],
  producer_contexts: [ISR | TASK | MAIN | DMA | UNKNOWN],
  consumer_contexts: [ISR | TASK | MAIN | UNKNOWN],
  confidence,
  notes
}

ChannelEdge {
  src_context,
  object_id,
  dst_context,
  edge_kind: DATA | CONTROL | MIXED,
  evidence_refs,
  score
}
```

Why this schema is useful:

- It is small and compositional: the linker can jump through object nodes without having to understand every possible RTOS primitive in detail.
- It matches the project's "LLM proposes, static verifies" philosophy: every node and edge can carry auditable evidence_refs instead of opaque heuristic scores only.
- It also maps naturally into the existing facts-bundle style used for BinAgent integration.

5.3 Construction algorithm (recommended first version)

5. Step 1 — Build context sets: use the existing ISR tagging plus a minimal main/task baseline (symbolic task entry if present; otherwise unknown/non-ISR).
6. Step 2 — Group SRAM accesses into object candidates: start with coarse clusters (for example 0x100 or 0x200 bytes), then split when evidence shows separate head/tail/flag vs payload arrays.
7. Step 3 — For each object candidate, compute writer and reader sets. Create an object node only if the object crosses a context boundary (for example ISR→non-ISR or DMA→CPU).
8. Step 4 — Detect special cases: small scalar flag, DMA object, and symbolic queue handle.

9. Step 5 — Rank consumer functions by relevance: prefer functions with parsing activity, copy sinks, or repeated reads from the same object.
10. Step 6 — Emit graph edges with evidence and confidence. Do not force a single producer or consumer if several plausible ones exist; keep top-K candidates and let the linker decide later.

5.4 Reasons, benefits, and defects of this construction strategy

Choice	Why we choose it	Benefit	Defect / risk
Cluster SRAM first	Because stripped firmware often destroys source-level object names and struct types.	Robust against missing symbols and reused globals.	May over-merge unrelated objects and create noisy edges.
Cross-context requirement	Because we specifically want async communication objects, not all globals.	Cuts search space dramatically and aligns with the research question.	Some real channels are same-context (task→task) and will be deferred.
Keep top-K producers/consumers	Because forcing one best explanation too early is brittle.	Preserves recall for later sink-first linking.	Needs ranking to avoid combinatorial explosion later.
DMA as a first-class object kind	Because DMA is common and cannot be seen as normal CPU stores.	Lets the system recover a major Type II/III source family correctly.	Controller-specific heuristics may miss complex descriptor-based setups.

5.5 Concrete examples of ChannelGraph output

Example A — ISR flag + payload buffer:

```
ISR(spi_ready_isr)  --CONTROL-->  flag(rx_ready)   --CONTROL-->  TASK(hci_spi_rx_thread)
TASK(hci_spi_rx_thread) --DATA--> object(hdr/payload cluster) --DATA--> TASK(hci_spi_rx_thread)
```

Example B — DMA channel:

```
DMA(config site)  --DATA-->  dma_buf  --DATA-->  TASK(net_task)
ISR(DMA_IRQHandler) --CONTROL--> dma_done --CONTROL--> TASK(net_task)
```

Example C — Ring buffer between ISR and main loop:

```
ISR(UART_IRQHandler) --DATA--> rx_buf/head --DATA--> MAIN(loop_consumer)
MAIN(loop_consumer) --DATA--> tail/parse state --DATA--> memcpy(dst, ..., len)
```

6. Sink-first source→sink tunnel detection (why, how, and trade-offs)

The chain detector should not enumerate all source×sink pairs. That would be a combinatorial mistake. The evaluation report already shows source counts in the thousands and sink counts in the thousands on a 48-sample suite. A full Cartesian product is the wrong abstraction. The right abstraction is sink-first linking: start from a sink, recover only the arguments that matter, and jump across ChannelGraph edges when ordinary slicing reaches an async boundary.

6.1 Why sink-first is the right default

Approach	What it means	Main benefit	Main defect

Full source×sink enumeration	Try all candidate sources against all candidate sinks.	Conceptually simple.	Explodes immediately in realistic corpora; expensive and noisy.
Source-first expansion	Grow forward from sources until sinks are found.	Good for coverage and discovering private/non-API sinks.	Very easy to over-taint and flood the graph with irrelevant paths.
Sink-first linking (recommended)	Start from a sink argument and trace backward; jump across channels only when needed.	Best precision/cost trade-off; naturally mirrors vulnerability reasoning and Mango-like workflows.	May miss nonstandard sinks if sink mining is weak, so it depends on good sink candidates.

Why this choice fits the current project state:

- SourceAgent is already good at finding firmware-specific sources. The missing capability is not more sources; it is proving relevance to dangerous arguments.
- The sink label set is already a compact set of "effect sites", so it is a natural place to start targeted reasoning.
- This follows the spirit of Mango: begin from potentially dangerous sites and search backward heuristically instead of modeling the entire program state space.

6.2 The tunnel-aware linking algorithm

11. Step 1 — Choose sink roots. For COPY/MEMSET/LOOP_WRITE, the primary root is len or loop bound; secondary roots are dst pointer and index. For FUNC_PTR, the root is the indirect target or dispatch index. For FORMAT_STRING, the root is the format argument.
12. Step 2 — Run a bounded backward slice inside the current function and, if necessary, a small number of caller hops. Stop when the slice reaches one of three boundary types: (a) a known source, (b) a channel object, or (c) budget exhaustion.
13. Step 3 — If the slice reaches a channel object, perform a tunnel jump: choose top-K producer sites from ChannelGraph, substitute the producer write value as the new root, and continue slicing on the producer side.
14. Step 4 — Along the way, recover parse/derive facts whenever the root becomes a value extracted from a buffer or global object (for example len = buf[2] | buf[3] << 8).
15. Step 5 — In parallel, summarize checks that dominate the sink. Mark them as effective, weak, or absent.
16. Step 6 — Emit a chain with verdict: CONFIRMED when a source is reached and check is absent/weak; SUSPICIOUS when sink is strong but source proof is incomplete; SAFE or DROP when a strong dominating bound is present.

6.3 Worked example: CVE-2020-10065 as a tunnel

Representative chain:

```
MMIO_READ(SPI1_DR)
  -> fills hdr[] / payload[]
  -> payload_len = parse_len(hdr)
  -> net_buf_add_mem(dst, payload, payload_len)
  -> missing or weak bound on payload_len
```

How the algorithm would traverse it:

- Root selection: start from COPY_SINK at net_buf_add_mem, choose payload_len as the primary root because it controls the size of the dangerous write.
- Backward slice inside the task: recover that payload_len is assigned from hdr[2] and hdr[3] (parse/derive fact).
- Backward from hdr reads: observe that hdr is filled from SPI1_DR reads. If this happens in the same task, the source is reached directly; if an ISR or callback fills a shared object, the slice will instead reach a channel object and tunnel through it.
- Check summarization: inspect whether a dominating clamp such as if (payload_len <= MAX) exists before the sink. If not, classify the chain as a likely vulnerability candidate.

Why this example is valuable:

- It shows why Type II/III cannot be reduced to syscall-based source modeling. The source is an MMIO FIFO, not recv().
- It shows why parse/derive matters: the dangerous argument is not the raw byte stream; it is the derived length field.
- It shows why channel modeling matters: in realistic drivers, the producer and consumer may be split across contexts.

6.4 Additional examples that should guide implementation

Example A — loop-copy in a main-loop consumer:

```
while (tail != head) {
  uint8_t n = rx_buf[tail++];
  for (i = 0; i < n; i++) {
    dst[i] = rx_buf[tail++];
  }
}
```

- Why this matters: there may be no memcpy call to detect; the sink is an inlined loop. This is exactly why sink mining cannot stop at APIs.
- What the linker needs: identify n as the loop bound root, then trace n to the shared receive buffer.
- What can go wrong: if loop analysis is weak, the code may look like a benign parser instead of a dangerous copy. This is a good candidate for optional LLM triage later.

Example B — function-pointer dispatch:

```
cmd = rx_buf[0];
handlers[cmd](ctx);
```

- Why this matters: the sink is control-flow, not memory copy. The root is the dispatch index cmd, not len.
- What the linker needs: trace cmd back to input and look for a dominating bound such as if (cmd < NUM_HANDLERS).
- Limitation: if the handler table is built dynamically or obscured behind wrappers, the first implementation may need to stop at a SUSPICIOUS verdict rather than CONFIRMED.

6.5 Major benefits and major defects of sink-first tunnel detection

Aspect	Benefit	Defect / caution
Precision	Focuses on few variables that	Can miss vulnerabilities when

	actually matter for exploitability (len/index/fmt/target).	sink mining is incomplete or overly narrow.
Scalability	Avoids full-program taint explosion and source×sink enumeration.	Still needs careful budgeting when a sink can tunnel into many producer candidates.
Actionability	Produces evidence chains close to what an analyst would write by hand.	Requires good explanations for why a tunnel jump was made; otherwise debugging becomes hard.
Compositionality	Works naturally with object-level ChannelGraph edges and verifier obligations.	Overly coarse object clustering can create false producer candidates.

7. Where should the linker live: SourceAgent, BinAgent, or both?

This is not just an implementation detail. It affects how experiments are run, how quickly the team can show progress, and which repository becomes the "home" of the end-to-end story.

Option	What it means	Why it is attractive	Why it is not enough by itself
A. SourceAgent-only linker	Add ChannelGraph and tunnel-aware linking as a new SourceAgent stage.	Makes the semantic recovery pipeline self-contained and publication-friendly for RQ1→RQ4.	Duplicates some logic that BinAgent already conceptually wants (sink-first reasoning and task queue analysis).
B. BinAgent-only linker	Keep SourceAgent as a fact/label layer; do all chain reasoning in BinAgent preflight.	Fastest path to a real vulnerability demo because it reuses the "analyze sink" mentality already present in BinAgent.	Leaves SourceAgent conceptually incomplete as a standalone research artifact.
C. Hybrid (recommended)	Keep ChannelGraph as a SourceAgent artifact, but consume it in BinAgent first. Later back-port a minimal linker stage to SourceAgent for evaluation.	Best trade-off between engineering speed and a clean research narrative.	Requires careful schema design so both repos speak the same language.

Recommended division of labor:

- SourceAgent should own: ContextIndex baseline, ChannelGraph generation, source/sink labels, evidence packs, and verifier-backed local facts.
- BinAgent should initially own: sink-first chain search, verdicting (CONFIRMED/SUSPICIOUS), and prioritization of which chains deserve deeper analysis.
- Later, a minimal linker can be re-imported into SourceAgent so that chain-level evaluation becomes reproducible in one place.

8. Where LLM should help, and where it should not

LLM should not replace the deterministic fact layer. It should be reserved for exactly the parts that human analysts currently solve by “reading the decompiler and summarizing semantics” — the places where hard rules become brittle and costly to maintain.

8.1 Good LLM jobs in this pipeline

Subproblem	Why rules struggle	What LLM can contribute	How to keep it safe
Parse/derive summarization	Length/type/index extraction appears in many arithmetic and state-machine forms.	Summarize which bytes/fields determine len/type/index and explain the parse logic.	Require evidence refs and verify the mentioned defs/uses with slices.
Check adequacy	A check may exist syntactically but still be semantically weak or unrelated to the sink.	Judge whether the check truly bounds the dangerous argument or target object.	Keep final decision in verifier/deep-check; treat LLM as triage, not truth.
Ambiguous loops	Copy loops, serialization loops, and benign initialization loops can look similar at IR level.	Classify suspicious loops into copy-like, walk-like, or benign categories and explain why.	Only run on top-ranked suspicious loops, then verify key claims with IR evidence.
Channel type naming	A cluster may be a ring buffer, a queue-like object, or just a shared state struct.	Offer a human-readable interpretation that helps debugging and reporting.	Do not let the interpretation create edges by itself; edges still come from static evidence.

8.2 Bad LLM jobs in this pipeline

- Do not ask LLM to invent MMIO sources. Peripheral-range reasoning and address provenance are exactly the sort of deterministic facts that static obligations are good at.
- Do not ask LLM to replace xref/callsite recovery for ordinary APIs when a proper IR or reference-manager solution exists.
- Do not let LLM create final vulnerability verdicts without static cross-checks. The project’s trust model depends on fail-closed verification.

8.3 Recommended “prefilter + LLM triage + verifier” workflow

17. Pass 1 — Prefilter (deterministic): mine all sources and sinks, build ChannelGraph, and generate suspicious sink candidates with local evidence.
18. Pass 2 — Rank suspicious candidates: prioritize those with variable len/index/format, weak/no obvious check, cross-context channel involvement, or loop-based writes outside known library code.
19. Pass 3 — LLM triage: only for the top-K suspicious cases, ask the model to summarize parse logic, classify the sink subtype, and comment on check adequacy.
20. Pass 4 — Verifier/deep-check: re-check every LLM claim against slices, dominance, object ranges, and optional bounded deep checks.

Why this workflow is good:

- It keeps costs low because only a small suspicious set is sent to the model.
- It preserves reproducibility because the underlying graph and slice facts remain deterministic.
- It creates a human-readable narrative for advisor meetings, papers, and debugging without giving up fail-closed guarantees.

Main risk:

- The team may become tempted to use LLM where engineering should fix the underlying static analysis. The line should stay firm: LLM explains or prioritizes, but IR facts still decide.

9. Recommended planning roadmap (P0–P3)

This roadmap is designed to maximize useful progress quickly. It prioritizes artifacts that will immediately improve the quality of real vulnerability chains, not just isolated labels.

Priority	Deliverable	Why now	Validation target
P0	ChannelGraph MVP (shared SRAM + DMA objects)	This is the smallest missing piece that unlocks cross-context reasoning.	At least one real CVE chain and two microbench chains can be recovered end-to-end.
P0	Sink-first tunnel linker for COPY/LOOP_WRITE roots	This directly turns current labels into vulnerability candidates.	Recover len/index roots back to sources on selected GT cases.
P1	Parse/derive miner MVP	Needed to explain why an input byte becomes a dangerous len/index.	Correctly identify len-from-buffer on microbench and CVE-2020-10065-like cases.
P1	Check summarizer MVP	Needed to separate truly risky chains from guarded copies.	Correctly label guard present/weak/absent on curated microbench cases.
P2	Queue/workqueue symbolic channels	Useful for broader RTOS coverage after the shared-memory MVP proves value.	Recover a small Zephyr/FreeRTOS queue example with strong symbol evidence.
P3	LLM triage on suspicious chains	Helps with semantic cases once the deterministic base is in place.	Reduce manual triage load without increasing unverified claims.

9.1 P0 in more detail: what should be built first

21. Task P0.1 — Extend ContextIndex just enough for the MVP. ISR tagging already exists; add a lightweight task/main baseline instead of chasing full RTOS recovery from day one.

22. Task P0.2 — Build ChannelGraph over existing MAI access clusters. Start with SRAM-crossing objects and verified DMA buffers.
23. Task P0.3 — Implement root-aware sink linking for COPY_SINK and loop-copy/loop-write cases. Focus on len and index first; defer complex pointer aliasing.
24. Task P0.4 — Demonstrate one real chain on CVE-2020-10065 and at least two microbench chains (for example a ring-buffer case and a DMA-length case).

Why this ordering is best:

- It produces a visible, advisor-friendly improvement quickly: not just "we found source and sink", but "we recovered the chain".
- It avoids overcommitting to hard RTOS-specific recognition before the core graph abstraction has been validated.
- It creates the foundation that later lets parse/check summaries matter. Without a chain backbone, parse and check facts have nowhere to attach.

9.2 Suggested evaluation plan for the new modules

Module	Best first dataset	What to measure	Why this metric matters
ChannelGraph MVP	Microbench + 1–2 CVE reproductions	Object precision, number of useful producer→consumer edges, manual sanity checks	A graph that is dense but wrong will hurt the linker more than having fewer high-quality edges.
Tunnel linker	Microbench with chain GT, then CVE-2020-10065	Chain recall, number of hops, source-reach success, runtime per sink	Shows whether the sink-first strategy remains tractable.
Parse/derive miner	Microbench + selected real examples	len/index derivation accuracy	This is the semantic bridge from raw bytes to dangerous arguments.
Check summarizer	Guard-heavy microbench	Guard present/absent/weak accuracy	Needed to reduce false positives once chains appear.
LLM triage	Only ambiguous chain subset	Agreement with human judgment, verifier pass rate, token cost	Prevents "LLM everywhere" from becoming uncontrolled.

9.3 Concrete success criteria

25. Success criterion 1 — The system can emit a chain of the form SOURCE → CHANNEL → DERIVE → CHECK? → SINK on at least one real CVE and several microbench samples.
26. Success criterion 2 — The chain detector runs sink-first and does not perform source×sink exhaustive pairing.
27. Success criterion 3 — The graph and linker remain explainable: every edge and jump has evidence refs that can be shown in a report or advisor meeting.
28. Success criterion 4 — The first version improves actionability even before it improves raw recall, i.e., the output becomes "fewer, better explained cases".

Related work and how this plan differs

The works below are the most relevant neighbors for the next phase. The point is not to cite everything in firmware analysis, but to explain exactly what design ideas we are borrowing and where our problem remains distinct.

Work	Core idea	Why it is relevant here	What it does not solve for us
Operation Mango (USENIX Security 2024)	Sink-first static data-flow analysis for taint-style bugs in Linux-based binary firmware services.	It strongly motivates starting from sinks and tracing backward instead of doing full top-down exploration.	It assumes a Type-I / service-style world with better process/I/O structure and does not solve Type II/III async context channels.
KARONTE (IEEE S&P 2020)	Track taint across multiple firmware binaries using modeled IPC interactions.	Conceptually similar to our ChannelGraph: both treat communication objects as the graph backbone.	Its “communication boundary” is cross-binary IPC, not ISR/task/main shared-state channels.
SaTC (USENIX Security 2021)	Leverages shared input keywords to locate data entry in Type-I firmware and then checks taint-style bugs.	Useful as a reminder that source discovery can be heuristic and domain-guided.	Its entry-point assumptions are web/service oriented; hardware MMIO/DMA sources are out of scope.
P2IM (USENIX Security 2020)	Dynamic peripheral modeling for firmware execution and fuzzing.	Good baseline for why MMIO/interrupt sources matter in MCU firmware.	Execution enablement is not the same as static chain reconstruction, and it does not provide an async source→sink graph by itself.
DICE (IEEE S&P 2021)	DMA-aware emulation for dynamic firmware analysis.	Validates that DMA channels are real and important enough to deserve first-class modeling.	It is emulator-centric, while our immediate problem is static graph recovery.
Fuzzware (USENIX Security 2022)	Precise MMIO modeling for effective monolithic firmware fuzzing.	Shows the value of precise hardware-input modeling and hardware-value usage.	Still primarily dynamic and does not solve static cross-context chain explanation.
RTCON (NDSS 2026)	Context-adaptive function-level fuzzing for RTOS kernels.	Important because it highlights how deeply context determines reachability in RTOS code.	It is a dynamic/fuzzing approach, not a static async graph and chain recovery system.
HEAPSTER (IEEE S&P 2022)	Analyzes allocators inside monolithic firmware.	Shows that domain-specific, deep semantic analysis can work on monolithic images.	It targets allocator bugs rather than general source→sink chains.
FirmXRay (CCS 2020)	Domain-specific static analysis of bare-metal BLE firmware.	Proof that static binary analysis can recover high-level semantics from bare-metal firmware.	Domain-specific and not a general async communication framework.

LATTE (ACM TOSEM 2025)	LLM-powered static binary taint analysis.	Relevant for using LLM as a taint-spec / semantic assistant rather than a standalone detector.	It still assumes the binary taint structure is already recoverable and is not specialized for Type II/III async channels.
IRIS (ICLR 2025)	Neuro-symbolic static analysis using LLMs to infer taint specs and filter paths.	A good model for “LLM proposes, static analysis verifies”.	It is source-code / repository oriented, not monolithic firmware / MMIO / ISR oriented.
AdaTaint (2025 preprint)	Adaptive LLM-driven source/sink inference with symbolic validation.	Good inspiration for “prefilter + LLM triage + validation” as an architecture pattern.	Still not targeted at hardware-driven inputs and async embedded semantics.

Closing recommendation

The project should now shift from “better isolated labels” to “better connected chains”. The single most important new artifact is a sparse, evidence-backed ChannelGraph, and the single most important new algorithm is a sink-first tunnel-aware linker. This combination is the smallest step that can turn today’s strong RQ1 label layer into a true Type II/III vulnerability-discovery layer. LLM should be introduced only after the deterministic backbone is in place, and only for the semantic cases that continue to resist stable rule engineering.

References and source documents

[I1] RQ1 Detailed Planning: Semantic Recovery v1.3 (internal proposal). Used for the label taxonomy, vertical slices, obligation design, and intended BinAgent integration.

[I2] SourceAgent update_0305.md (internal weekly update). Used for the current framing of context boundaries, the CVE-2020-10065 walkthrough, and the “what remains to build” list.

[I3] eval_suite_unstripped_elf_report_en.md (internal evaluation report). Used for the current implementation snapshot and the scale argument against source×sink enumeration.

[R1] Gibbs et al. Operation Mango: Scalable Discovery of Taint-Style Vulnerabilities in Binary Firmware Services. USENIX Security 2024.

[R2] Redini et al. KARONTE: Detecting Insecure Multi-binary Interactions in Embedded Firmware. IEEE S&P 2020.

[R3] Chen et al. Sharing More and Checking Less: Leveraging Common Input Keywords to Detect Bugs in Embedded Systems. USENIX Security 2021.

[R4] Feng et al. P2IM: Scalable and Hardware-independent Firmware Testing via Automatic Peripheral Interface Modeling. USENIX Security 2020.

[R5] Mera et al. DICE: Automatic Emulation of DMA Input Channels for Dynamic Firmware Analysis. IEEE S&P 2021.

[R6] Scharnowski et al. Fuzzware: Using Precise MMIO Modeling for Effective Firmware Fuzzing. USENIX Security 2022.

[R7] Lee et al. RTCON: Context-Adaptive Function-Level Fuzzing for RTOS Kernels. NDSS 2026.

[R8] Gritti et al. HEAPSTER: Analyzing the Security of Dynamic Allocators for Monolithic Firmware Images. IEEE S&P 2022.

[R9] Wen et al. FirmXRay: Detecting Bluetooth Link Layer Vulnerabilities from Bare-Metal Firmware. CCS 2020.

[R10] Liu et al. LLM-Powered Static Binary Taint Analysis (LATTE). ACM TOSEM 2025.

[R11] Liu et al. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. ICLR 2025.

[R12] Lin. AdaTaint: LLM-Driven Adaptive Source-Sink Identification and False Positive Mitigation for Static Analysis. 2025 preprint.